

Algorithm XXXX: truesum — Exact Elementwise Summation of Floating-Point Matrices on CPUs and GPUs

truesum: Exact Elementwise Summation of Floating-Point Matrices

NICHOLAS WILT

Archaea Software, LLC, United States, nicholas@archaeasoftware.com

Floating-point addition is not associative, so a sum of double-precision values depends on the order of evaluation, and parallel or distributed summations may vary from run to run. `truesum` is a C++17 library that sums floating-point matrices elementwise and exactly. Every entry of an accumulation matrix is a two's complement fixed-point integer wide enough that no rounding occurs, making the result independent of submission order, thread count, and device. Each column shares one exponent, placed at the least significant bit of any of its values rather than the most significant, and storage is limb-major, so that one entry's carry chain runs within a vector lane while rows fill the lanes. Unlike accumulators whose width is a property of the format, a column is sized from the dynamic range of its data. A twelve-byte survey per column, performed on input matrices, records the lowest true unit in the last place and the highest occupied bit and allows an accumulation matrix to be sized before any input matrices arrive. Readback is correctly rounded, optionally with an exact residual, and a mean over a caller-supplied count is rounded once rather than twice.

CCS CONCEPTS • Mathematics of computing → Mathematical software • Mathematics of computing → Arbitrary-precision arithmetic • Computing methodologies → Parallel algorithms

Additional Keywords and Phrases: Exact summation, reproducibility, block floating point, superaccumulator, fixed-point accumulation, correct rounding, AVX-512, CUDA, limb-major storage

ACM Reference Format:

Nicholas Wilt. 2026. Algorithm XXXX: truesum — Exact Elementwise Summation of Floating-Point Matrices on CPUs and GPUs. ACM Trans. Math. Softw. 0, 0, Article 0 (2026), 14 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>
Author's address: Nicholas Wilt, Archaea Software, LLC, United States; email: nicholas@archaeasoftware.com; ORCID: [to be supplied]. This work received no external funding. An earlier and substantially different draft of this material was posted by the author to a personal newsletter; it has not been published or submitted elsewhere.

1 INTRODUCTION

Floating-point operations notoriously are not associative. In particular, $(a+b)+c \neq a+(b+c)$, introducing a degree of uncertainty about the accuracy of workloads that rely on floating-point summation.

To eliminate that uncertainty, our library `truesum` provides an *accumulation matrix* that can perform fast, elementwise exact summation of floating-point matrices. Each column is represented as block floating point, with a shared exponent and the mantissas held in a structure-of-arrays (SOA) layout of 64-bit *limbs*, GMP's term for the segments of a multi-precision number that each fit in a single machine word [Granlund and the GMP Development Team 2023]. Storage is per limb position rather than per element: one contiguous array holds limb k of every row, and we call such an array a *limb-column*, for want of a better term.

A *survey* is a compact (12 bytes per column) characterization of an input matrix that may be used to pre-size an accumulation matrix to receive its contents without overflow, rescaling, or the allocation of new limb-columns during the accumulation. Extending this idea, an accumulation matrix can be pre-configured for multiple input matrices by supplying one survey for each prospective input matrix.

A final feature of the accumulation matrix is that it can return the mean, exactly rounded, an operation that is both faster and more accurate than requiring the client to divide a readback by the number of accumulations.

Both AVX-512 and CUDA implementations are provided, and both saturate the bandwidth on their respective platforms. Batched submissions conserve bandwidth to the accumulation matrix on both platforms, and the CUDA implementation optionally takes a stream parameter so API clients can coordinate concurrent data movement and accumulation.

1.1 Prior Work

Exact summation is not a new idea, and the fixed-point accumulator is its oldest form. Kulisch's *complete register* takes the approach to its limit: an accumulator spanning the entire exponent range of the format, 4288 bits wide for dot products of `doubles`, of which 88 are headroom so that up to 2^{88} products can be added without rounding or overflow [Kulisch 2008; Kulisch and Snyder 2011a; 2011b]. Neal's superaccumulators do the same in software, sixty-seven 64-bit chunks overlapping by 32 bits so that carries need not propagate on every add [Neal 2015]. ExBLAS pairs a 2098-bit superaccumulator with floating-point expansions held in registers, and takes both onto GPUs [Iakymchuk et al. 2015; Collange et al. 2015]. ReproBLAS gives up exactness for reproducibility alone: it splits each addend into slices along a predefined grid of 40-bit bins and keeps only the three bins at and below the largest addend seen so far, so its accumulator occupies six `doubles` [Demmel et al. 2016; Ahrens et al. 2020].

A recent study that swapped ExBLAS and ReproBLAS, along with a third library, OzBLAS, into distributed Krylov solvers illustrated the tradeoffs between these two choices [Lei et al. 2025]. ReproBLAS was on average the fastest of the methods tested, ahead even of an ordinary MPI reduction, but on the more ill-conditioned dot products its relative error reached order one, as an ordinary sum's did. ExBLAS stayed correctly rounded throughout, at 9 to 29 times the ordinary cost per iteration.

Kulisch, Neal and ExBLAS share an accumulator whose width is a property of the format rather than of the data: 4288 bits, whether the values span four binades or four hundred. Demmel and Nguyen's first reproducible sum sized itself from the data, computing $\max|x_i|$ before adding anything and setting its rounding boundary from that maximum and the count [Demmel and Nguyen 2013; 2015]. Demmel et al. [2016] call that the simplest approach and reject it on cost: "The trouble with this simple approach is that it requires 2 or 3 passes over the data, or 3 communication steps in parallel." The predefined bins of ReproBLAS are how they got to one pass. `trueSum` keeps the pass and moves it: each column is sized to the dynamic range it actually holds, and surveys, computed by whichever device already has the data, enable those columns to be pre-sized before accumulation begins.

2 STRUCTURE OF ARRAYS

The SOA layout enables both AVX-512 and CUDA implementations to process summations at the speed of memory bandwidth.

Within the matrix, a single element is stored as a two's complement integer spanning several 64-bit limbs. Written the usual way (most significant first), it reads `[ limb 2 | limb 1 | limb 0 ]`. Each limb position has its own

separate, contiguous allocation, so limbs 0, 1 and 2 for a given element are stored at the same offset into three different limb-columns (Figure 1).

Due to the dynamic range of `double`, with its 11-bit exponent, the worst-case space requirement for a single column of `double` is an eye-popping 33 limbs, or 264 bytes per entry. At risk of stating the obvious, the library was designed with an eye toward better-conditioned inputs that drive more modest requirements.

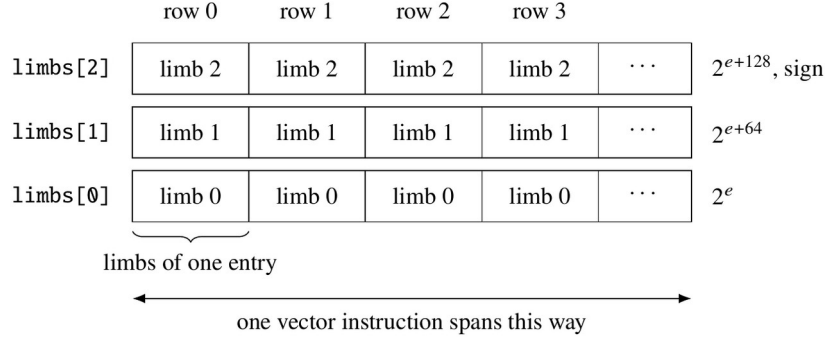


Fig. 1. Limb-major storage. Each `limbs[k]` is a limb-column: one allocation holding limb k of every row. A vector instruction spans rows; the carry chain of one entry runs down its column of limbs.

Propagating carries along the limbs of a single entry is inherently sequential: limb $k+1$ cannot be computed until after limb k . Rows, by contrast, are completely independent of one another, so the limb-major memory layout enables rows to be processed by a single vector instruction (8 64-bit elements at a time on AVX-512), while the sequential carry chain becomes the loop over limb positions, one vector instruction per position.

3 PER-COLUMN BLOCK FLOATING POINT

On the assumption that columns contain similarly scaled values, the library implements a block floating-point scheme that shares an exponent per column: not the exponent of its largest value, but the position of the *least significant* bit of any of its values. This arrangement stands in contrast to IEEE standard representations, whose exponents identify the position of the *most significant* set bit. Recall that floating-point values have more than one exact decomposition into an integer times a power of two: 1.0 is exactly 1×2^0 , but just as exactly $2^{52} \times 2^{-52}$. IEEE 754 uses the latter, because the most significant set bit is redundant: with the exception of denormals, the format encodes a hidden bit to increase its effective precision. 1.0 is the tidy case, a mantissa of 1 meaning the value is a power of two; nothing about odd normalization reduces every mantissa to a single bit. 3.0 is exactly 3×2^0 , and its 53-bit form is $6755399441055744 \times 2^{-51}$, so `frexp` implies a last place fifty-one positions below the value's own rather than fifty-two.

The power of two specified by a column's exponent, then, is each value's true unit in the last place, or ulp. Taking the minimum over true ulps rather than over standard exponents can save up to 52 bits per column: a column of small integers stays at exponent 2^0 and one limb wide, whereas a `frexp`-based scale would reduce it to 2^{-52} , forcing the allocation of a second limb holding nothing but zeros. Figure 2 shows the bit positions of a three-limb entry.

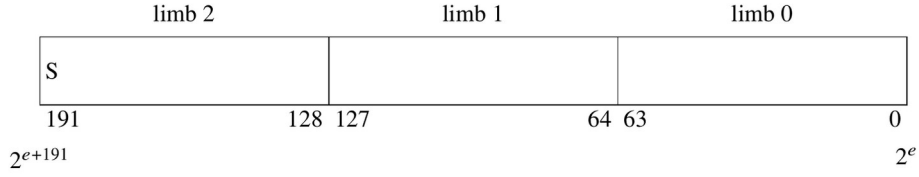


Fig. 2. One entry across three limbs. Bit 0 of limb 0 has significance 2^e , where e is the column's exponent; the top bit of the top limb is the sign.

A column maintains four parameters that must be updated with a survey before applying an accumulation: **exponent**, **max_addend_bits**, **add_count**, and the number of limbs. **exponent** can only decrease; the other three only increase. None of them changes on a pre-sized accumulation matrix: when the surveys of every incoming matrix are supplied at construction, **exponent** and the limb count are set once from those extents, **max_addend_bits** and **add_count** are never touched at all, and each accumulate skips the survey, the rescale, and the widen.

max_addend_bits is the greatest number of bits any addend in the column has spanned, counting from the column's current bit 0 up to that addend's most significant bit. **max_addend_bits** and **add_count**, a running count of the addends, both are updated from a survey of the incoming values, taken before any of those values is added. A rescale adjusts **max_addend_bits** too, adding the shift it applied, since moving the column's exponent down by that many bits widens every addend's span from the new bit 0 by the same amount. Both exist to bound the width the accumulated sum can reach, so a column whose entries are all zero needs neither. **max_addend_bits** and **add_count** are cleared when the caller empties the accumulation matrix for reuse, and when a rescale finds every entry already zero, which is also the one case where the exponent can change without shifting anything.

exponent decreases whenever a value arrives whose least significant set bit sits below the column's current **exponent**. Every entry is then left-shifted to match: existing bits move, and none are discarded.

An increase in the limb count, and allocation of one or more new limb-columns, may be prompted by any of three events:

- a value whose most significant bit sits above the high water mark denoted by **max_addend_bits**;
- a lower exponent, since that prompts a rescale of **shift** bits; and
- more addends, since a longer run can sum to a larger total.

The sum of these considerations determines the number of bits needed for a column:

$$\text{bits} = \text{max_addend_bits} + \lceil \log_2(\text{add_count} + 1) \rceil + 1, \quad \text{limbs} = \lceil \text{bits} / 64 \rceil \quad (1)$$

max_addend_bits accounts for the largest single addend; the logarithmic term is headroom reserved above it, gaining one bit each time the count doubles, so that **add_count** addends cannot carry past it; and the final bit is reserved for the sign, which no sum may reach. Rounding the total to the next multiple of 64 gives the number of limbs needed for the allocation.

A batch of input matrices is surveyed before any are accumulated: for any given column, a first pass over the incoming values determines whether the column must be rescaled and widened before any values are added.

Reallocation and/or rescaling of a column may be prompted by either very large or very small values. Consider a column with 1.0, which sets **exponent** = 0 and allocates a single limb. Adding a large value to this existing column

does not change `exponent`, but if more limbs are needed, the newly allocated limbs are initialized with a sign extension (all-zero or all-one copies of the most significant bit). Adding an extremely small value decreases `exponent`, also increases the limb count (2^{-500} drops `exponent` to -500 and widens the column to 8 limbs; 10^{-300} decreases `exponent` to -1049 and widens the column to 17 limbs), and requires rescaling of the existing limb-columns, left-shifting their contents by exactly the amount `exponent` moved and shifting zeros into the least significant bits. Figure 3 contrasts the two adjustments.

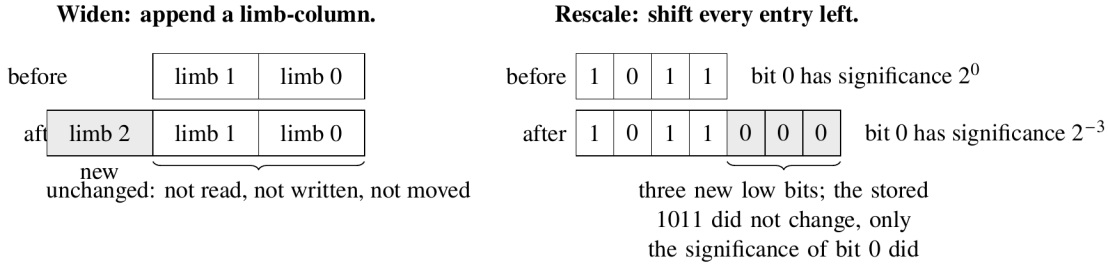


Fig. 3. The two adjustments. A widen appends a limb-column and leaves every existing limb-column untouched. A rescale lowers the column's exponent and shifts every stored value left by that amount, so the value each entry denotes is unchanged and nothing is lost.

A `double` arrives as a 53-bit odd mantissa placed at `shift`, the distance from its own true ulp down to the column's. Since $53 < 64$, a `double` may span at most two limbs, so only `off` and `off+1` need to be updated, with limbs above `off+1` receiving carry propagation until it is absorbed (Figure 4).

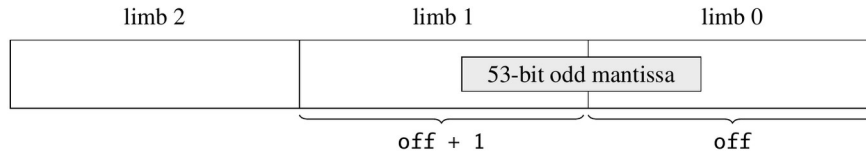


Fig. 4. An addend lands in at most two adjacent limbs. Its lowest bit sits `shift` positions above the column's bit 0; `off` = $\lfloor \text{shift}/64 \rfloor$ selects the lower limb and `shift mod 64` the bit offset within it.

The carry chain runs fully vectorized, without an overflow test until it reaches the most significant limb, where a sign comparison of the operands is performed against the result.

A column need only allocate as many limb-columns as needed for the dynamic range spanned by its own data. In the bundled demonstration program, a 4×4 accumulation matrix holding values spanning roughly 10^{-30} to 10^{24} occupies 5720 bytes, with per-column allocations of two and three limbs rather than the thirty-three that covering the whole `double` range at once would require.

4 SURVEYS

Surveys supply the per-column metadata needed to know how many limb-columns are needed, and whether the column must be rescaled before the survey's matrix is submitted for accumulation.

A survey is 12 bytes per column and contains four fields:

- `min_exponent`, the lowest true-ulp exponent among the column's nonzero values, taken after normalizing each mantissa to odd rather than from `fexp`, so a column of small integers reports 0 and not -52 ;

- `max_top`, one past the highest bit position any value occupies;
- `any`, true if any value was nonzero;
- `nonfinite`, true if any value was infinite or NaN.

Both exponents are `int` rather than `long long`, since 32-bit exponents can represent six orders of magnitude more range than `double` can produce.

Computing a survey requires at most two sweeps over the column (Algorithm 1). The first masks off the sign bit, early-outs if any value is infinite or NaN, and computes the maximum of the remaining values, which later is used to compute `max_top`. Additionally, the minimum biased exponent is computed, clamped at 1 so every denormal reports the same one. That minimum is a lower bound on the true ulp: a biased exponent locates the lowest place a significand could occupy, while the true ulp sits above it by the significand's trailing zero count. Unless the column held nothing but zeros, or a non-finite value ended the first sweep, or that bound is larger than a cutoff supplied by the caller, a second sweep of the column is needed to find the significance of the mantissa's least significant set bit and take the minimum in order to compute `min_exponent`.

```

1: any ← false; nonfinite ← false; b ← +∞; m ← 0
2: for all  $x_i$  do                                     ▷ sweep 1: raw bits only
3:   if  $x_i$  is infinite or NaN then nonfinite ← true; return
4:   end if
5:   if  $x_i \neq 0$  then
6:     any ← true; b ← min(b, max(biased( $x_i$ ), 1) − 1075); m ← max(m,  $|x_i|$ )
7:   end if
8: end for
9: if not any then return
10: end if
11: max_top ← 1 +  $\lfloor \log_2 m \rfloor$                          ▷ exact, from the bit pattern of m
12: if b ≥ c then min_exponent ← b; return             ▷ no rescale can be due
13: end if
14: min_exponent ← +∞
15: for all  $x_i \neq 0$  do                               ▷ sweep 2: locate the least significant set bit
16:   min_exponent ← min(min_exponent, biased( $x_i$ ) − 1075 + ctz(significand( $x_i$ )))
17: end for

```

Algorithm 1. Survey of one column $x_1, \dots, x_n$ of `doubles`, given a cutoff *c*.

An accumulation matrix constructed without surveys computes them itself, one per incoming matrix, and in that case the second sweep is often unnecessary because the accumulation matrix already knows each column's exponent, which it passes as the cutoff. Any incoming value below the column's exponent causes the column to be rescaled; a first-sweep bound at or above the cutoff rules that out on its own. The width does not depend on the second sweep, either: `max_addend_bits` is `max_top` less the column's exponent, and the first sweep computes `max_top` exactly.

A producer computing a survey to send elsewhere has no column to compare against, so the second sweep is always performed. Neither does the accumulation matrix when it surveys a column that has no exponent yet: that column will adopt whatever the survey reports, and since a column's exponent only ever moves down, a bound would leave its scale below what its values require, for good.

Functions to update the accumulation matrix include an entry point for a single column, for a column-major matrix, and for a row-major one, which stages each column into contiguous memory. A fourth surveys a matrix already resident on the device, for a producer whose data never touches host memory at all.

Sizing a column from one survey takes its exponent from `min_exponent` and its span from `max_top - min_exponent`, then adds the headroom of Equation (1) for however many matrices are coming. Sizing from several surveys of the same column takes the minimum of the minima and the maximum of the maxima. Reducing them to a single aggregate is what makes submission order irrelevant: any matrix inside those extents fits the reservation, so the accumulation matrix never has to know which one is being submitted.

Two adjustments may follow from a survey, one for each extent it reports. A `max_top` above what the column's limb-columns can hold appends more limb-columns, each initialized with a sign extension, and leaves the existing values unchanged. A `min_exponent` below the column's `exponent` calls for a rescale: a fresh set of limb-columns is allocated, every stored value is shifted left by the difference with zeros filling the least significant bits, and `max_addend_bits` is increased by that same shift. The exponent specifies the significance of the lowest bit position the column can hold, so increasing it would right-shift every stored value, dropping bits already accumulated; it only ever decreases. An all-zero column is the exception: `max_addend_bits` and `add_count` reset to zero, since the total they bound is zero, and the exponent reaches its new value with nothing to reallocate and nothing to shift. It still only decreases, and the number of limb-columns allocated is not changed.

The separation of concerns between matrices and their surveys enables the work to be done by different constituencies (Figure 5). In a data center, the nodes that computed the matrices can also compute their surveys and transmit them ahead to the accumulation node; once the accumulation matrix there has been pre-sized to receive them all, the matrices themselves follow, one matrix or even one column at a time. The final output is bit-identical regardless of the order in which they arrive or are processed.

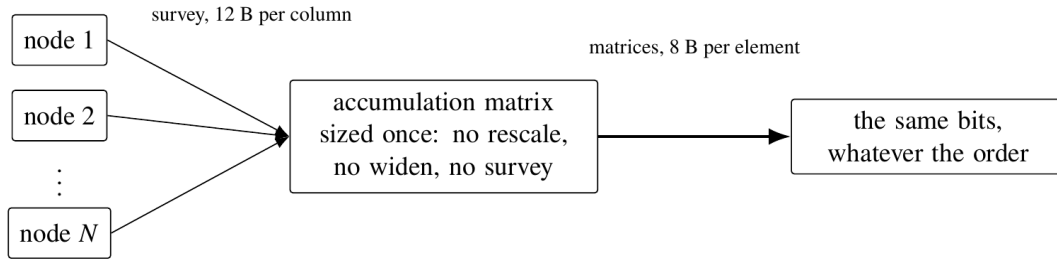


Fig. 5. Surveys travel ahead of the data. Once the accumulation matrix is sized from all of them, the matrices themselves may arrive and be accumulated in any order.

5 ACCUMULATION

Since a survey has been performed and applied to the accumulation matrix ahead of time, the accumulation kernel need not concern itself with rescaling or allocation of limb-columns. Every column already has the correct exponent and limb count when the kernel starts, so it can be updated in a single pass over the input (Algorithm 2). A value's lowest set bit sits some number of positions above the lowest position the column can hold. That distance, the value's true-up exponent minus the column's exponent, is the shift: divided by 64, it gives the limb where the addend starts, and the remainder gives its bit offset within that limb. A 53-bit significand occupies at most two limbs at any bit offset, and the carry propagates upward from the lower limb.

```

1: for all rows  $i$ , eight at a time on AVX-512 do
2:    $(m, t, s) \leftarrow$  odd mantissa, true-ulp exponent and sign of  $x_i$  ▷ skip if  $x_i = 0$ 
3:    $shift \leftarrow t - e$ ;  $off \leftarrow \lfloor shift/64 \rfloor$ ;  $bit \leftarrow shift \bmod 64$ 
4:   flag kBadExponent if  $shift < 0$ ; flag kBadWidth if the addend's top reaches the sign bit
5:    $a_{off} \leftarrow m \ll bit$ ;  $a_{off+1} \leftarrow m \gg (64 - bit)$ ; negate the pair if  $s$  ▷ two-limb addend
6:   for  $k = off$  to  $L - 1$  do ▷ carry chain, one vector op per limb
7:      $(limbs[k][i], carry) \leftarrow limbs[k][i] + a_k + carry$ 
8:     if  $carry = 0$  and  $k > off + 1$  then break
9:     end if
10:  end for
11:  flag kBadOverflow if the top limb's sign disagrees with both operands'
12: end for

```

Algorithm 2. Accumulate one column of addends into a column with exponent e and L limbs.

A column allocated wide for its dynamic range therefore costs no more per addend than a narrow one, except as required by the input values.

On a pre-sized accumulation matrix, the exponents and limb counts were set at construction from the surveys, which are verified as the accumulation kernel runs: three tests on an addend before it is applied, and a fourth on the running total as it crosses into the top limb.

Between them, those tests detect any deficiencies in the surveys. A survey can understate either of a column's extents, or fail to report a non-finite value; separately, a caller can supply fewer surveys than the matrices it goes on to submit. Each of those throws `std::runtime_error` rather than quietly returning an incorrect sum. They are designed to err on the side of caution: a total that wraps past the sign bit and later wraps back is correct by modular arithmetic, but is reported anyway.

An addend below the column's exponent or too wide for it is skipped; a non-finite one is recorded and applied anyway; and an overflowed sum is left where it wrapped. The rest of the batch goes in either way, and the error is raised afterwards, by which point the accumulation matrix is inconsistent. What it is never left in is a torn state: the kernel loads an entry's limbs, applies whole addends to them, and writes them all back together, so an addend is either absent or present in full.

If no surveys are provided, the accumulation matrix will compute one before beginning the accumulation.

The accumulation matrix only ever touches one column at a time, and accordingly, the caller can submit individual columns for accumulation. This interface is useful when matrices are large and columns are much smaller, so only one column and the accumulation matrix have to be resident. A positive or negative power-of-two scale factor also may be specified, and the shift is applied before adding the input into the limb-column.

A symmetric $n \times n$ matrix can be accumulated into one stored triangle. That halves the memory and the input traffic, and the CPU time falls with it, but the deeper reason is exactness. In full storage, the two halves would hold equal values in different limbs under different column scales. Stored once, it cannot drift. The input is assumed to be symmetric.

Accumulation can be spread over worker threads, partitioned by column. Columns are independent in storage and each is touched by exactly one worker, so there is no locking on the hot path and the result is bit-identical to a serial run. Scaling is limited by the CPU's external memory bandwidth.

To conserve bandwidth to the accumulation matrix, batches of input matrices may be accumulated in a single pass: the limbs are read once and every addend is applied in registers before writing the limbs back to memory.

6 READING BACK THE RESULT

Once the accumulation matrix is computed, the library provides the following services:

- Extraction of its contents at `double` precision, correctly rounded: round-to-nearest, ties-to-even, over the full range. `to_double` returns one entry and `to_matrix` the whole thing. A sum beyond the range of a `double` reads as $\pm\text{inf}$. One that lands in the denormal range is rounded directly to a multiple of 2^{-1074} , rather than twice, first to 53 bits and then again on the way into the denormals.
- Extraction of the *residual*, the portion of each matrix element that rounding dropped, itself correctly rounded. The subtraction is done in the matrix's own fixed point, so the residual is the true difference and not a floating-point estimate of it. It is at most half an ulp of the returned value and is negative exactly when the rounding went up, so the pair is a non-overlapping two-term expansion of about 106 bits. It is +0.0 when the value was representable, which causes `is_exactly_representable` to be set. Computing the residual is optional.
- Extraction of an entry in decimal form by `to_exact_decimal`, every digit, never rounded. A binary fixed-point value always has a terminating decimal expansion, since $V \cdot 2^{-k} = (V \cdot 5^k)/10^k$, so the expansion is finite however wide the sum. A single 0.1 accumulated once reads back as 0.10000000000000000055511151231257827021181583404541015625.
- Extraction of the *mean* over a count the caller supplies, correctly rounded as a quotient. Dividing the rounded sum would round twice; `to_double_mean` and `to_matrix_mean` round once, by long division of the exact sum, and offer the same residual overloads as the readbacks above. The count is the caller's because the accumulation matrix does not know what a sum stands for: a scaled add is a weighted one, and per-element, per-column and whole-matrix adds all feed the same cell. Neal [2015], whose superaccumulators began as a way to compute the sample mean in R, observes the same double rounding and proposes this single-rounding division as future work.

7 CUDA CONSIDERATIONS

When CUDA is available, developers can use `CudaAccumulationMatrix`, a separate class that keeps the accumulation matrix in device memory. In contrast to the CPU implementation, where the AVX-512 kernel is transparently utilized when the instruction set is available, developers must opt into use of `CudaAccumulationMatrix`; it is not an acceleration that gets switched on when CUDA is present. The CUDA limb arrays are allocated in device memory, and putting them behind the same interface would disguise host-to-device transfers as ordinary calls. What the two implementations share is the algorithm: when to rescale, when to widen, how the exponent moves. The kernels are bandwidth-bound throughout, running at very nearly the card's streaming rate, so performance is determined by how many times an input crosses the bus and how often the CPU and GPU wait on each other.

The constructor for `CudaAccumulationMatrix` takes an optional stream parameter which, if specified, is used for every kernel launched by the class—the survey, the accumulate, and the sign extension behind a widen and the shift behind a rescale—as well as the stream-ordered allocation and release of the limb-columns and the small copies

that bring column extents back to the host and send column descriptors out to the device. Absent a stream, the constructor creates one and destroys it along with the matrix; either way, `stream()` returns the one in use. Only a few setup copies stay off the stream, issued synchronously while a column is being allocated or widened.

The survey runs on whichever processor already holds the matrix. A producer holding a matrix in host memory surveys it on the CPU. A producer whose matrix never leaves the device, delivered by GPUDirect or computed there, calls `survey_matrix_col_major_device` instead.

With the extents known before the accumulate launches, the kernel reads each input exactly once, so the input matrix can be read straight out of mapped memory at bus-saturating speeds rather than first copying it to device memory. Input from the host must therefore be page-locked and device-mapped. Input matrices that are not accessible to the accumulation kernel via device memory or mapped pinned memory must be staged to such memory before invoking the kernel.

When reading matrix data from mapped pinned memory, the developer must take care to avoid write-after-read race conditions, since the kernel will still be streaming its input when the call returns. A submission is a kernel launch on the accumulation matrix's stream, so the synchronization may be accomplished by recording a CUDA event on the matrix's stream after updating the matrix. A producer with two buffers in rotation would keep one event per buffer and wait only for the one it is about to refill, never for the stream as a whole.

A producer can avoid the CUDA events altogether by queuing its copies on that same stream, which then orders each copy against the accumulate: a single buffer can be refilled in place, with nothing to wait on. A two-buffer rotation on a caller's stream costs little more per batch than the fill alone, so nearly all of the accumulate hides behind the transfer.

For input matrices in mapped pinned memory, the accumulation runs at bus-saturating bandwidth. Device-resident input is several times faster, and submitting input matrices in batches is faster still, because the limb-columns are read and written once per batch instead of once per matrix.

8 PERFORMANCE

Table 1 reports throughput measured on an 8-core AMD Ryzen 7 7700X and an NVIDIA RTX 3060, in billions of input values accumulated exactly per second. The driver programs that produce these figures are included with the software, so that the measurements can be repeated; the recommendations of Johnson [2002] on the experimental analysis of algorithms informed how they are reported, in particular the identification of the machine, the reporting of medians rather than best times, and the discussion below of how strongly the figures depend on the input data rather than on its dimensions alone.

Table 1. Exact accumulation throughput, in 10^9 input values per second, for column-major inputs of 64 columns. “Batched $K = 8$ ” submits eight matrices in one pass over the accumulation matrix.

	4096 × 64	16384 × 64	65536 × 64
CPU, 1 thread	0.80	0.77	0.67
CPU, 8 threads	4.31	4.65	1.11
CPU, 8 threads, batched $K = 8$			3.01
GPU, host input	3.11	3.26	3.32
GPU, input resident	4.40	6.07	8.13
GPU, resident, batched $K = 8$			25.3

Even in elements per second, the figure depends on the values and not only on the shape. A column's width comes from the dynamic range of its contents, so two matrices of identical dimensions can differ several-fold: values within a few binades of one another share a single limb, while a column reaching across the whole `double` range needs thirty-three. Each element costs 8 bytes of input against 16 bytes per limb of accumulation matrix, read and written, so the data sets the width and the width determines the cost. Any single number here describes the inputs it was measured on.

Learning the width has its own cost, and the two adjustments are not alike. Appending a limb-column writes one new array of `ROWS` limbs and reads the one below it to sign-extend from; a rescale allocates a fresh set and moves every limb-column the column already holds. Table 2 shows both on one CPU thread for a 65536×16 matrix: a submission needing neither ran 1.0 ms; one appending a limb-column ran 2.0 to 3.0 ms whatever the column's current width; one forcing a rescale ran 5.0 ms at two limbs and 12.0 ms at thirteen, rising with the width because the width is what it rewrites. That asymmetry is why `reserve_for_surveys` earns its place: a caller who knows roughly how low its data reaches pays for one rescale up front rather than one per submission that reaches lower than the last.

Table 2. Cost of one submission of a 65536×16 matrix on one CPU thread as the column grows, in milliseconds. Each widen step increases the largest addend by 2^{64} ; each rescale step lowers the smallest by 2^{-64} . A submission with nothing to adjust costs 1.0 ms.

limbs after	widen	rescale
2	3.0	5.0
4	2.9	5.8
6	2.9	6.5
8	2.7	6.8
10	2.5	8.6
13	2.2	12.0

On the device, the same two adjustments cost differently again, because they entail memory management, not just processing. Allocating a limb-column from the stream-ordered pool is close to free, at one to two microseconds once the pool has grown to size, against about three milliseconds for the first allocation. The bulk of performance impact comes in the form of needed synchronization. `grow_column` and `rescale_column` each drain the stream before touching the column's array of base pointers, and that array is allocated with plain `cudaMalloc` and released with `cudaFree`, neither of which is stream-ordered. A widen on the GPU is therefore a pipeline stall rather than a memory pass, taken once per column that needs one. Pre-sizing removes both calls entirely, which matters more here than on the CPU: the CPU pays a predictable pass over memory, while the device pays for however much work it must abandon mid-flight.

9 SOFTWARE

The accompanying software component contains the library, its test suite, the demonstration program, the benchmark drivers that produce Tables 1 and 2, and a user manual describing installation, dependencies and every public entry point. The library is C++17 with no dependencies beyond a C++ compiler and CMake; the AVX-512 kernels are compiled when the compiler supports them and selected at run time, and the CUDA class is built when a CUDA toolkit is found. The library builds and its tests pass on Linux and, in its portable scalar configuration, on macOS. Future versions of the software package will be made available via Wilt [2026].

10 CONCLUSION AND FUTURE WORK

The accumulation matrix offers accuracy and reproducibility: no matter which device, or how many cores or threads, or what order data arrives across the network, it produces the same bit-identical output because it utilizes integer arithmetic in a fixed point wide enough that never requires rounding. It bears repeating that exactness is not accuracy. A sum of `doubles` routinely needs more bits than any one of its addends, which is the whole reason the `residual` and `to_exact_decimal` exist, but no amount of exactness recovers information the inputs never held. The fifty-five digits that come back for a single accumulated 0.1 are every one of them correct, and every one of them a fact about the `double` nearest 0.1 rather than about the number the caller meant to add.

A survey is twelve bytes per column, small enough to serve as metadata that describes a matrix and enables future accumulations to be sized properly before accumulation begins. Pre-sizing accumulation matrices removes the need to rescale the existing data, or to append limb-columns partway through the accumulation.

Demmel and Nguyen's reproducible sum already began with a pass to find $\max|x_i|$, and both they and the ExBLAS authors treated that pass as overhead to be designed away [Demmel and Nguyen 2013; Iakymchuk et al. 2015]. A `truesum` survey records both extents, the lowest true ulp as well as the highest bit, because an exact sum cannot drop the low bits that otherwise would be rounded away. The node that already holds a matrix can compute its survey and send it ahead of the data. When the accumulation matrix surveys its own input instead, it does so a column at a time, so the `accumulate` usually finds that column still in cache. Surveys of any number of matrices reduce to one reservation by taking minima and maxima, and a reservation made that way is checked rather than trusted, since the `accumulate` tests every addend against the extents it was given.

An accumulation matrix holds one accumulator per entry, so the accumulator's size multiplies across every row and column. Demmel et al. [2016] show that the memory traffic of a tiled matrix multiply grows with the square root of the accumulator's size, which is why ReproBLAS accepts rounding error in order to limit its accumulator to six words. Columns sized from their surveys stay small without giving up exactness, to the extent the data allow.

As a reminder: *Exact accumulation restores associativity*, so whole-matrix, column-major, per-column and per-element accumulation all produce the same output regardless of order of submission; the threaded, batched, symmetric, pre-sized and GPU paths must each match the plain serial one; and the AVX-512 kernels are cross-checked against the scalar ones by forcing the portable path. Catastrophic cancellation ($10^{300} + 1 - 10^{300}$ yields exactly 1), a million values whose ulp is far below the running total, and every rounding tie from $1 + 2^{-53}$ down to the 2^{-1075} round-to-zero tie are checked individually. The test suite runs in well under a second and is clean under AddressSanitizer, UndefinedBehaviorSanitizer and ThreadSanitizer.

For future work, several directions suggest themselves.

- **Merging accumulation matrices.** Columns at the same exponent and width add limb-wise with a single carry chain, and columns at different exponents need the rescale already implemented in the library. A tree reduction requires this operation, and its absence is why the above distributed story contemplates sending every matrix to one accumulator rather than combining partial sums pairwise. Every accumulator cited in the introduction already provides one: Kulisch's complete addition, Neal's sum of two superaccumulators, the merge stages of ExBLAS and ReproBLAS's addition of one indexed sum to another.
- **Narrower input formats.** `float`, `half` and `bfloat16` all widen to `double` exactly, so they need no new kernel, only a conversion at the boundary. The performance benefits of this feature are expected to be greater for CUDA, where the `accumulate` is bandwidth-bound and a `half` input crosses the bus at a quarter the bytes.

- **Exact gemm through the Ozaki scheme.** Ozaki et al. [2012] split A by rows and B by columns into slices narrow enough that the product of any two slices, computed by an ordinary `gemm`, is exact, which turns AB into an unevaluated sum of floating-point matrices. Their paper leaves that sum to an accurate summation algorithm and stores every product matrix until then, which the authors call a drawback: working space significantly larger than `gemm`'s. An accumulation matrix can take each product as it is formed and sum all of them exactly, with no change to its kernels, since every addend is an ordinary `double`. The surveys come from the splitting itself. Every entry of a slice product is an integer multiple of $u^2\sigma_i\tau_j$ and at most $u\sigma_i\tau_j$ in magnitude, where u is the unit roundoff and σ_i and τ_j are the powers of two the splitting already chose for row i of A and column j of B , so each product can be submitted pre-sized and checked as it accumulates, provided no slice product underflows, which the scheme assumes as well. The scheme needs more slices as the dynamic range within a row or column grows, just as a `truesum` column needs more limbs.
- **Exact products without splitting.** The product of two `doubles` is exact in 106 bits, which a fixed-point accumulator holds without rounding, so an exact dot product can also be built by widening the addend instead. The products are formed on the fly, so none exists ahead of time to survey, but a product's extents are bounded by its factors': its true ulp is exactly the sum of theirs, and its top at most the sum of their tops, so the surveys of the two input matrices are enough to size the accumulation. The exponent range doubles with it, so the worst case grows accordingly.
- **A spill path for wide batches.** The AVX-512 batch kernel holds a row's limbs in registers, capping the batch at eight limbs; wider columns fall back to one matrix at a time and give up the batching saving exactly where columns are widest and it would be worth most.
- **binary128.** Every format above is narrower than `double` and rides the existing path. A wider one does not: 113 bits of significand breaks the two-limb placement that all three kernels are built around.

ACKNOWLEDGMENTS

Claude (Anthropic), used through Claude Code, assisted in preparing this work: it drafted and revised sections of the text under the author's direction, wrote and ran the benchmark programs whose measurements appear in Tables 1 and 2, located and summarized several of the cited works, and contributed to the library's source code. The author reviewed all generated material, verified the citations against their sources, and is responsible for the content.

REFERENCES

- Willow Ahrens, James Demmel, and Hong Diep Nguyen. 2020. Algorithms for efficient reproducible floating point summation. *ACM Transactions on Mathematical Software* 46, 3, Article 22 (2020), 49 pages.
- Caroline Collange, David Defour, Stef Graillat, and Roman Iakymchuk. 2015. Numerical reproducibility for the parallel reduction on multi- and many-core architectures. *Parallel Computing* 49 (2015), 83–97.
- James Demmel and Hong Diep Nguyen. 2013. Fast reproducible floating-point summation. In *Proceedings of the 21st IEEE Symposium on Computer Arithmetic (ARITH-21)*. IEEE, Austin, TX, 163–172.
- James Demmel and Hong Diep Nguyen. 2015. Parallel reproducible summation. *IEEE Transactions on Computers* 64, 7 (2015), 2060–2070.
- James Demmel, Willow Ahrens, and Hong Diep Nguyen. 2016. *Efficient Reproducible Floating Point Summation and BLAS*. Technical Report UCB/EECS-2016-121. EECS Department, University of California, Berkeley, CA.
- Torbjörn Granlund and the GMP Development Team. 2023. *GNU MP: The GNU Multiple Precision Arithmetic Library* (6.3.0 ed.). Section 3.2, Nomenclature and Types. Retrieved from <https://gmplib.org/manual/Nomenclature-and-Types>.
- Roman Iakymchuk, Caroline Collange, David Defour, and Stef Graillat. 2015. ExBLAS: Reproducible and accurate BLAS library. In *Proceedings of the Workshop on Numerical Reproducibility at Exascale (NRE at SC15)*. ACM, Austin, TX.
- David S. Johnson. 2002. A theoretician's guide to the experimental analysis of algorithms. In *Data Structures, Near Neighbor Searches, and Methodology*:

- Fifth and Sixth DIMACS Implementation Challenges*, Michael H. Goldwasser, David S. Johnson, and Catherine C. McGeoch (Eds.). DIMACS Series in Discrete Mathematics and Theoretical Computer Science, Vol. 59. American Mathematical Society, Providence, RI, 215–250.
- Ulrich Kulisch. 2008. *Computer Arithmetic and Validity: Theory, Implementation, and Applications*. de Gruyter, Berlin, Germany.
- Ulrich Kulisch and Van Snyder. 2011a. The exact dot product as basic tool for long interval arithmetic. *Computing* 91, 3 (2011), 307–313.
- Ulrich Kulisch and Van Snyder. 2011b. *The Exact Dot Product*. Proposal to the IEEE P1788 working group on interval arithmetic. Retrieved from <https://grouper.ieee.org/groups/1788/email/pdf7OYfgSd9H4.pdf>.
- Xiaojun Lei, Tongxiang Gu, Xiaowen Xu, and Stef Graillat. 2025. A general framework for reproducible parallel preconditioned Krylov methods using three BLAS variants. In *Proceedings of the 9th International Conference on High Performance Compilation, Computing and Communications (HP3C 2025)*. ACM, Jinan, China, 63–68.
- Radford M. Neal. 2015. *Fast Exact Summation Using Small and Large Superaccumulators*. arXiv:1505.05571. University of Toronto, Toronto, Canada.
- Katsuhisa Ozaki, Takeshi Ogita, Shin'ichi Oishi, and Siegfried M. Rump. 2012. Error-free transformations of matrix multiplication by using fast routines of matrix multiplication and its applications. *Numerical Algorithms* 59, 1 (2012), 95–118.
- Nicholas Wilt. 2026. *truesum*: source repository. <https://github.com/ArchaeaSoftware/parallelprogrammer/tree/main/truesum>.